

Python Libraries Lab

University Mohammed VI Polytechnic

College of Computing

Professor: Fahd KALLOUBI

Course: CS3 - MLOps

Academic Year: 2025/2026

Submission Date: November 2025

MAHMOUD MAFTAH

INTRODUCTION:

This lab report covers the practical implementation of **Pytest**, **Pydantic**, and **FastAPI** for moving machine learning models from research to production.

1. Introduction to Pytest

1.1 Basic Testing

Implementation:

- Created my_module.py with a square(x) function
- Implemented test_my_module.py with basic assertions
- Pytest automatically detected files starting with test_
- Verified test failures by changing expected values

Key Observations:

- Pytest discovers tests automatically without configuration
- Clear failure messages improve debugging
- Test files must follow naming convention (test_*.py)

First test:

```
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\m\Lab folder - FastAPI Pydantic\Pytest
plugins: anyio-4.11.0
collected 1 item

test_my_module.py . [100%]

===== 1 passed in 0.03s =====
```

After changing value 4 to 7:

```
platform win32 -- Python 3.14.0, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\m\Lab folder - FastAPI Pydantic\Pytest
plugins: anyio-4.11.0
collected 1 item

test_my_module.py F [100%]

===== FAILURES =====
test_square_gives_correct_value

  def test_square_gives_correct_value():
      subject= square(2)
      > assert subject == 1
E       assert 4 == 1

test_my_module.py:5: AssertionError
===== short test summary info =====
FAILED test_my_module.py::test_square_gives_correct_value - assert 4 == 1
===== 1 failed in 0.15s =====
```

Objective: Use fixtures to share setup code across tests.

- Created fixtures with `@pytest.fixture` decorator
- Implemented `input_value()` fixture returning test data
- Used fixtures as function parameters in test functions
- Centralized fixtures in `conf/test.py` for reusability

- Eliminates code duplication across tests
- `conftest.py` makes fixtures available to all test files
- Cleaner test code with separation of setup and assertions

```
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\m\lab folder - FastAPI Pydantic\Pytest
plugins: anyio-4.11.0
collected 1 item

test_my_module2.py . [100%]

===== 1 passed in 0.03s =====
```

```
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\m\Lab folder - FastAPI Pydantic\Pytest
plugins: anyio-4.11.0
collected 2 items

test_my_module2.py .
test_my_module_again.py .

===== 2 passed in 0.03s =====
```

```
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\m\Lab folder - FastAPI Pydantic\Pytest
plugins: anyio-4.11.0
collected 2 items

test_my_module2.py . [ 50%]
test_my_module_again.py . [100%]

===== 2 passed in 0.03s =====
```

1.3 Parametrized Tests

Objective: Test multiple inputs with single test function.

Implementation:

- Used @pytest.mark.parametrize decorator
- Tested multiple values [2, 3, 4] with one test function
- Each parameter generated a separate test case

Results:

- Converted 1 test function into 3 executed tests
- Efficient way to test multiple scenarios
- Total tests increased from 2 to 4 in the suite

Paramaterized test:

```
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\m\Lab folder - FastAPI Pydantic\Pytest
plugins: anyio-4.11.0
collected 4 items

test_my_module2.py . [ 25%]
test_my_module_again2.py ... [100%]

===== 4 passed in 0.03s =====
```

2. Data Validation with Pydantic

2.1 Basic Models

Objective: Understand Pydantic's data validation and type coercion.

Implementation:

- Created User model inheriting from BaseModel
- Defined fields with type annotations (int, str, List, Optional)
- Tested automatic type conversion (string '123' → integer 123)
- Verified datetime parsing from string format

Key Observations:

- Mandatory fields require type annotations without defaults
- Optional fields accept None or use default values
- Automatic type coercion when possible
- ValidationError raised for invalid data (e.g., non-numeric strings for int fields)

2.2 Recursive Models

Objective: Create complex nested data structures.

Implementation:

- Created nested models (Foo, Bar, Spam)
- Used models as types within other models
- Demonstrated composition with lists of model instances

Result: Successfully validated complex hierarchical data structures, proving Pydantic's capability for real-world API schemas.

Example 1:

```
123
datetime.datetime(2019, 6, 1, 12, 22)
[1, 2, 5]
D:\m\Lab folder - FastAPI Pydantic\pydantic\ex1.py:19: PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated in Pydan
tic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
  print(user.dict())
{'id': 123, 'name': 'John Doe', 'signup_ts': datetime.datetime(2019, 6, 1, 12, 22), 'friends': [1, 2, 5]}
```

Types were converted for id '123' string to int 123, '5' string to int 5, signup_ts string to datetime

Example 2:

```
foo=Foo(count=4, size=0.0) bars=[Bar(apple='x1', banana='y'), Bar(apple='x2', banana='y')]
D:\m\Lab folder - FastAPI Pydantic\pydantic\ex2.py:23: PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
print(m.dict())
{'foo': {'count': 4, 'size': 0.0}, 'bars': [{'apple': 'x1', 'banana': 'y'}, {'apple': 'x2', 'banana': 'y'}]}
```

2.3 Custom Validators

Objective: Implement business logic validation rules.

Implementation:

- Created UserModel with password matching validation
- Implemented validators for:
 - Name format (must contain space)
 - Username format (alphanumeric only)
 - Password confirmation matching

Key Features:

- @validator decorator for field validation
- Access to other field values via values parameter
- Clear error messages for validation failures
- Validators can transform data (e.g., .title() for names)

Validator:

```
D:\m\Lab folder - FastAPI Pydantic\pydantic\ex3.py:10: PydanticDeprecatedSince20: Pydantic V1 style `@validator` validators are deprecated. You should migrate to Pydantic V2 style `@field_validator` validators, see the migration guide for more details. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
    @validator("name")
D:\m\Lab folder - FastAPI Pydantic\pydantic\ex3.py:17: PydanticDeprecatedSince20: Pydantic V1 style `@validator` validators are deprecated. You should migrate to Pydantic V2 style `@field_validator` validators, see the migration guide for more details. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
    @validator("password2")
D:\m\Lab folder - FastAPI Pydantic\pydantic\ex3.py:24: PydanticDeprecatedSince20: Pydantic V1 style `@validator` validators are deprecated. You should migrate to Pydantic V2 style `@field_validator` validators, see the migration guide for more details. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
    @validator("username")
name='samuel colvin' username='scolvin' password1='zxcvbn' password2='zxcvbn'
```

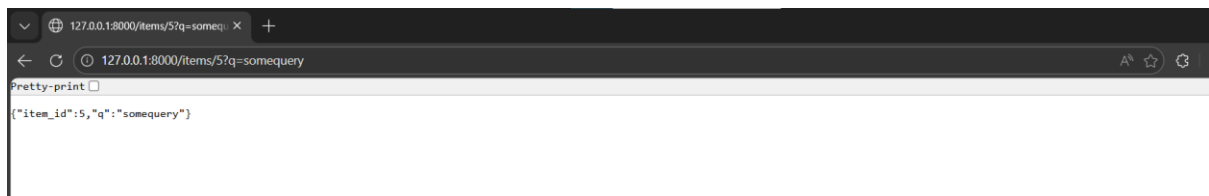
3. Building APIs with FastAPI

3.1 Introduction to FastAPI

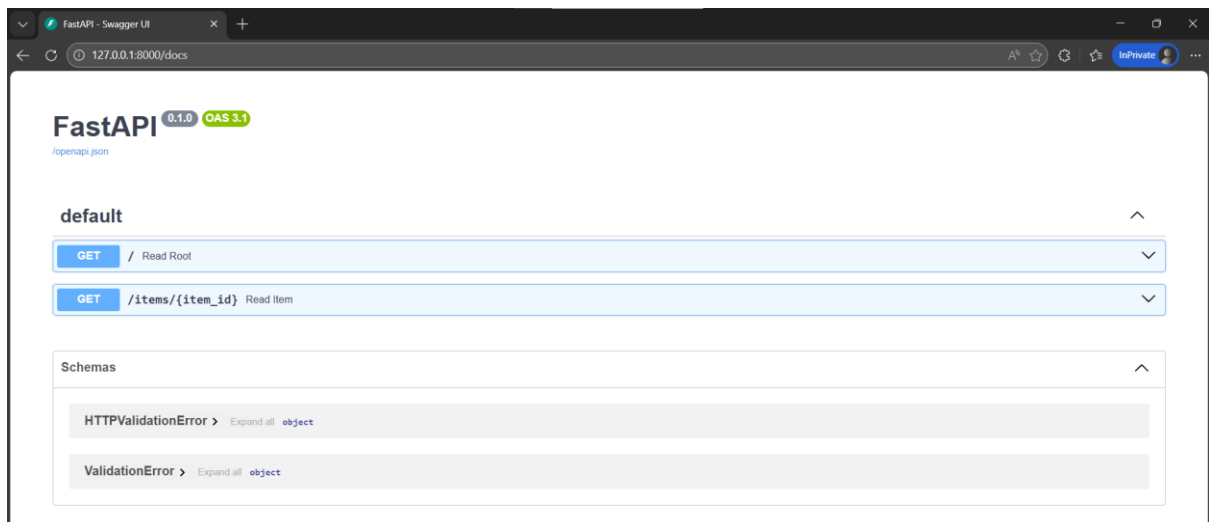
Advantages over Flask:

- Built-in data validation via Pydantic
- Automatic API documentation (Swagger UI)
- Asynchronous support with ASGI server
- Type hints for better IDE support and error detection
- Production-ready with minimal configuration

JSON RESPONSE FROM LINK



VALIDATION ERRORS



TEST AFTER ID CHANGE

The screenshot displays the Swagger UI for a FastAPI application. The endpoint `GET /items/{item_id}` is selected, with the description "Read Item".

Parameters:

- `item_id` (integer, path): Required, with a text input field.
- `q` (string, query): Optional, with a text input field.

Responses:

Code	Description	Links
200	Successful Response Media type: <code>application/json</code> Example Value: <code>"string"</code>	No link
422	Validation Error Media type: <code>application/json</code> Example Value: <code>{ "detail": [{ "loc": ["item_id", "q"], "msg": "string", "type": "string" }] }</code>	No link

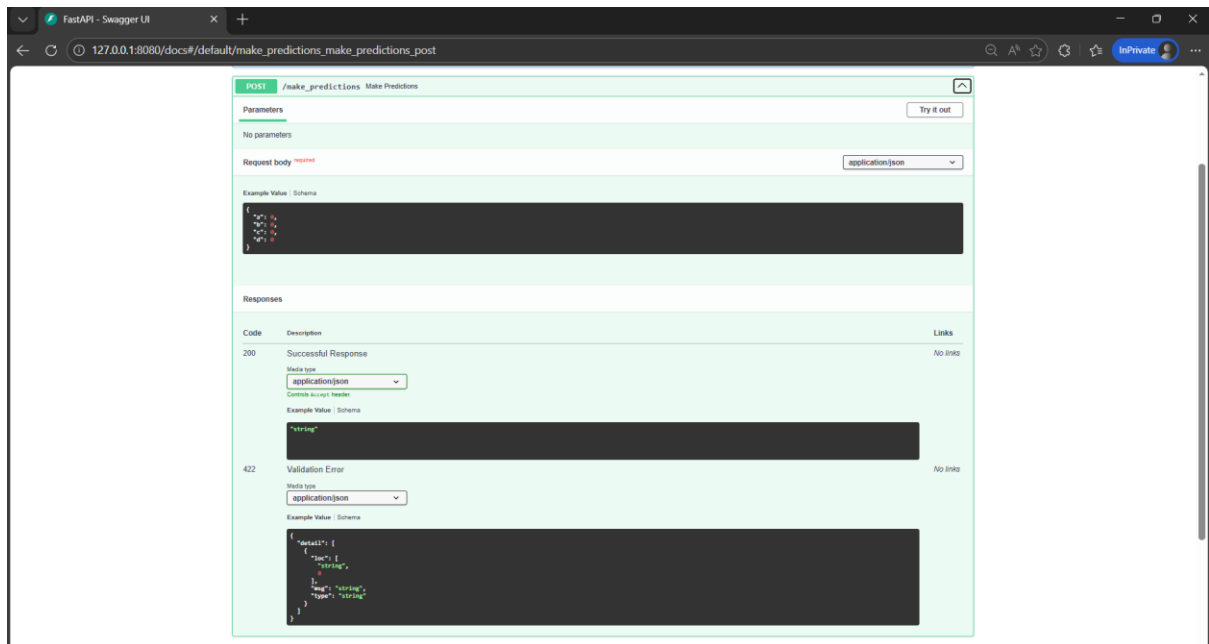
The screenshot shows the FastAPI Swagger UI home page. The title is "FastAPI 0.109.0 GIN 2.16.3".

default

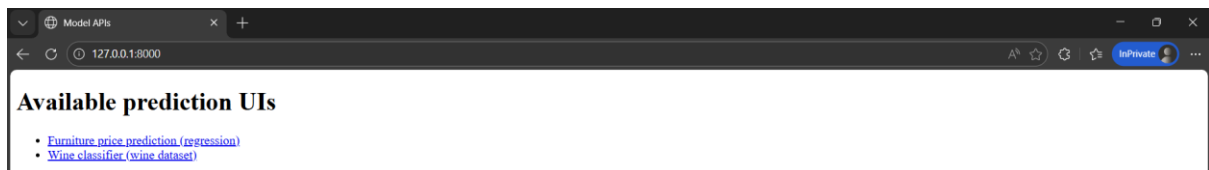
- `GET /` Home
- `POST /make_predictions` Make Predictions

Schemas

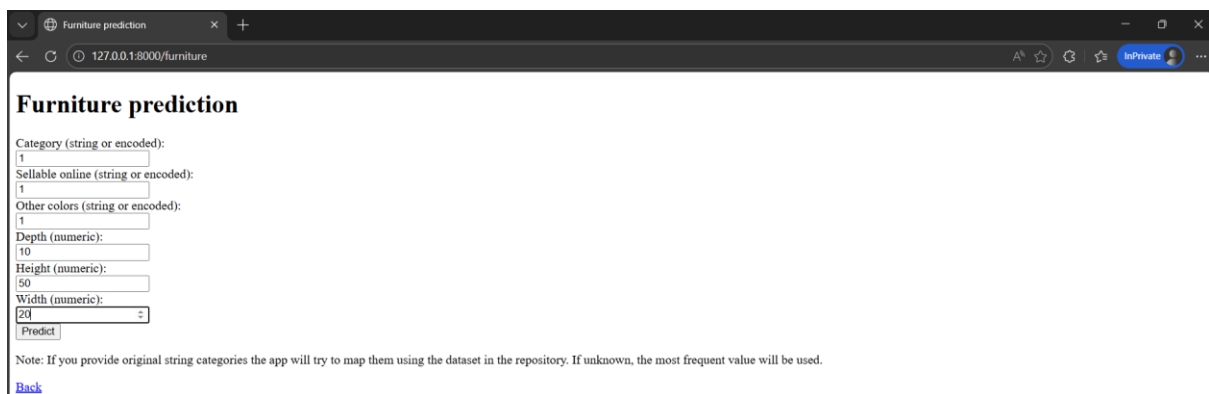
- `HTTPValidationError` Expand all object
- `ValidationError` Expand all object
- `Iris` Expand all object



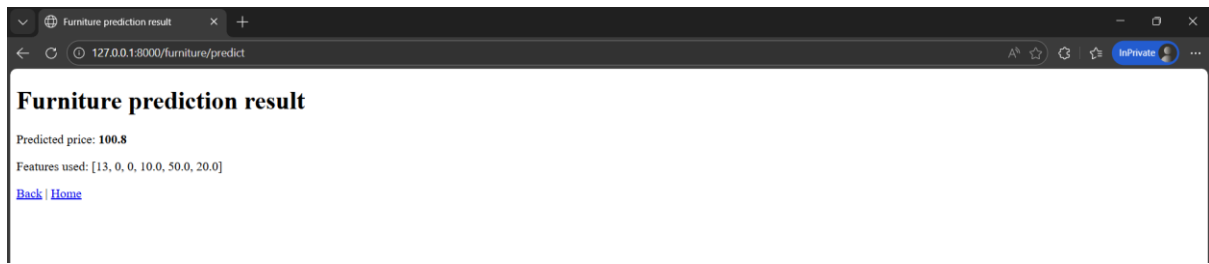
EXPLORING AVAILABLE PREDICTION UIs:



TESTING THE FURNITURE PREDICTION:



FURNITURE PREDICTION RESULTS:



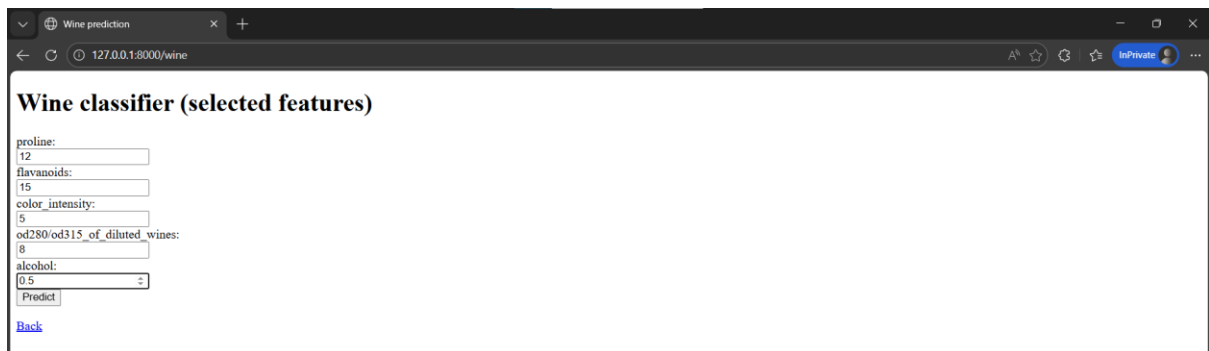
Furniture prediction result

Predicted price: **100.8**

Features used: [13, 0, 0, 10.0, 50.0, 20.0]

[Back](#) | [Home](#)

WINE CLASSIFIER TEST FROM THE SET OF SELECTED FEATURES



Wine classifier (selected features)

proline:

flavanoids:

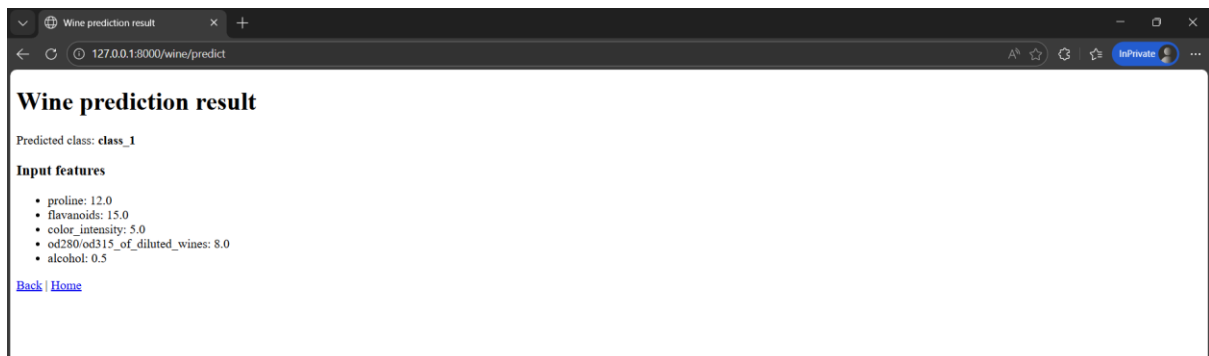
color_intensity:

od280/od315_of_diluted_wines:

alcohol:

[Back](#)

WINE PREDICTION RESULTS



Wine prediction result

Predicted class: **class_1**

Input features

- proline: 12.0
- flavanoids: 15.0
- color_intensity: 5.0
- od280/od315_of_diluted_wines: 8.0
- alcohol: 0.5

[Back](#) | [Home](#)